**Memory**

Memory is the main component of every system that stores data and instructions. A computer's memory is mainly categorized into two types: primary and secondary memory.

# What is Primary memory?

The ***primary memory*** of a PC system is its primary memory. The instructions that must be performed at the time are copied to primary memory because the CPU may directly access data from the primary memory. Accessing data from this memory is faster because it is the internal memory, and the CPU accesses data from primary memory via the data bus.

Primary memory is typically volatile in nature, which implies that information in primary memory doesn't exist if the information is not stored in the event of a power loss. It is more costly than secondary memory because it is made of semiconductors, and primary memory capacity in a computer is restricted and is always less than secondary memory.

## Types of Primary Memory

***RAM (Random Access Memory)*** and ***ROM (Read Only Memory)*** are the two types of primary memory.

**1. RAM (Random Access Memory)**

RAM is both a read and writes type of memory. The information that has to be currently processed is stored in RAM, which the CPU may immediately access. It is a volatile memory that loses data when the electricity is turned off. RAM may be static or dynamic in nature.

**2. ROM (Read Only Memory)**

ROM is a read-only memory, which implies that its contents may not be altered. It contains the instructions that are executed when the system boots up. It is a non-volatile memory, which implies that it preserves its contents even after the power is turned off. It has three types of ROM, such as PROM, EPROM, and EEPROM.

## Features of Primary Memory

There are various features of Primary Memory. Some important features of Primary Memory are as follows:

1. It is faster to access when compared to secondary memory.
2. If the power is turned off, you may lose data.
3. It is the computer's working memory.
4. It is known as primary memory.
5. Without primary memory, the system cannot function.
6. It is also known to as volatile memory.

# What is Secondary Memory?

***Secondary memory*** is additional computer memory. Secondary memory stores data that must be preserved permanently. The CPU cannot directly access secondary memory data, and the data must first be copied to primary memory before the CPU can process it. As a result, accessing data from secondary memory takes longer. The secondary memory may be accessed via the input-output channel.

It is non-volatile memory, which implies that its content remains even after the power is turned off. It is optical or magnetic memory, which is less expensive than primary memory.

Secondary memory is always more plentiful than primary memory. As an external memory, a computer may even function without secondary memory. Secondary memory includes hard disks, CDs, DVDs, floppy disks, etc.

## Types of Secondary Memory

There are various types of secondary memory. Some of them are as follows:

**1. Floppy Disk**

It is a magnetic disk that is enclosed in a square plastic box. Its main goal is to store information and then move it from one system to another. Floppy disks come in two sizes (1) Size: 3.5 inches,

storage capacity: ***1.44 MB (2) 5.25-inch*** display and ***1.2-MB storage capacity***. A system contains a floppy disk drive to use a floppy disk. DVDs, CDs, and flash drives have mostly replaced this type of storage media.

**2. Hard Disk**

Hard disks are storage devices that may hold a large amount of data. Hard disks are often known as hard disk drives. It comes in a range of storage capacities (For instance, ***256 GB, 500 GB, 1 TB, and 2 TB***). It is constructed of platters, which are collections of disks. The plates are piled on top of one another and covered in a magnetic coating. Each plate is composed of a series of invisible circles, each with an identical core track. Hard disks are classified into two types: internal and external.

**3. SD Card**

It is a type of memory card that is often utilized to store data in portable devices like cameras and cell phones. It is available in several capacities, such as ***1, 2, 4, 8, 16, 32, and 64 GB***. The data on the SD card can be accessed by removing it from the system and placing it into a system through a card reader. The SD card data is stored in memory chips, and there are no moving parts, as there are on a hard disk.

## Features of Secondary Memory

There are various features of Secondary Memory. Some important features of Secondary Memory are as follows:

1. These are optical and magnetic memories.
2. It is a sort of non-volatile memory.
3. It aids in the storage of data in a PC.
4. The data is stored permanently, even when the system is turned off.
5. The system can function without secondary memory.
6. It is slower than the primary memory.

| Feature | Primary Memory | Secondary Memory |
|---|---|---|
| Definition | Volatile memory located on CPU | Non-volatile memory for long-term data storage |
| Speed | Fast | Slow |
| Cost | Expensive | Inexpensive |
| Capacity | Limited | Large |
| Volatility | volatile (data is lost when power is turned off) | Non-volatile(retains data even after power is turned off) |
| Access Time | Fast | Slow |
| Purpose | Stores data and instructions temporarily for quick access by the CPU | Stores data and programs for long-term storage and retrieval |
| Examples | Dynamic RAM (DRAM), Static RAM (SRAM) | Hard Disk Drive (HDD), Solid State Drive (SSD), Optical Disk Drive (ODD), USB Flash Drive, Memory Card |
| Used as | Main memory | Backup and archival storage |
| Uses | Holds data and instructions temporarily while they are being processed | Stores data and programs for long-term use |

**Identifiers**
Identifiers are user defined word used to name of entities like variables, arrays,functions, structures etc.
**Rules for naming identifiers are:**
1) name should only consists of alphabets (both upper and lower case), digits and underscore (_) sign.
2) first characters should be alphabet or underscore
3) name should not be a keyword
4) since C is a case sensitive, the upper case and lower case considered differently, for example code, Code, CODE etc. are different identifiers.
5) identifiers are generally given in some meaningful name such as value, net_salary, age, data etc. An identifier name may be long, some implementation recognizes only first eight characters, most recognize 31 characters.

**Keyword**
There are certain words reserved for doing specific task, these words are known as reserved word or keywords. These words are predefined and always written in lower case or small letter. These keywords can't be used as a variable name as it assigned with fixed meaning. There are 32 keywords in C **Some examples are int, short, signed,unsigned, default, volatile, float, long, double, break, continue, typedef, static, do, for, union, return, while, do, extern, register, enum, case, goto, struct, char, auto, const etc.**

**Operator**
This is a symbol use to perform some operation on variables, operands or with the constant.
Several operators are there those are, arithmetic operator, assignment, increment , decrement, logical, conditional, comma, size of , bitwise and others

## Arithmetic Operators

These operators perform basic mathematical operations like addition, subtraction, multiplication, and division. We will categorize the arithmetic operators into two categories based on the number of operands and then look at their functionality.

| Type | Operator | Name of Operator | Functionality |
|---|---|---|---|
| Unary | ++ | Increment | Increases the value by 1 |
| | - - | Decrement | Decreases the value by 1 |
| Binary | + | Addition | Add two values |
| | - | Subtraction | Subtracts one value from the other |
| | * | Multiplication | Multiplies two values |
| | / | Division | Divides one by the other value |
| | % | Modulus | Finds the remainder after division |

# Relational Operators

They are also known as Comparison Operators. They compare the values of the two operands. The result of the comparison is either `true` or `false`. If the comparison is true, it returns 1; If the comparison results in false, it returns 0. These are known as boolean values.

| Operator | Name |
|---|---|
| == | Equal to |
| > | Greater than |
| < | Less than |

| | |
|---|---|
| >= | Greater than or equal to |
| <= | Less than or equal to |
| != | Not equal to |

## Logical Operators

They are used to combine two or more conditions/constraints. It returns either 0 or 1 depending upon whether the expression results in true or false. If the result is `true`, it returns `1` else returns `0`. The logical operators are used in decision-making and looping statements.
We will categorize the logical operators into two categories based on the number of operands and then look at their functionality.

| Type | Operator | Name | Functionality |
|---|---|---|---|
| Binary | && | Logical AND | returns 1(true) if both the expressions/values are true. |
| | \|\| | Logical OR | returns 1(true) if one of the expressions/values evaluates to true. |
| Unary | != | Logical NOT | Negates the expr |

## Bitwise Operators

These operators work on individual bits. The operands are first converted into bits i.e. 0 or 1, and then the calculation is performed on them.
We will categorize the bitwise operators into two categories based on the number of operands and then look at their functionality.

| Type | Operator | Name |
|---|---|---|
| Unary | ~ | One's complement or Bitwise Complement |
| | << | Left Shift |
| | >> | Right Shift |
| Binary | & | Bitwise AND |
| | \| | Bitwise OR |
| | ^ | Bitwise Exclusive OR or XOR |

## Assignment Operators

These are used to assign values to the variables. The most fundamental assignment operator is "=".
The following table shows some variants of the assignment operator, "=".

| Operator | Example | Same as |
|---|---|---|
| = | x=y | x=y |
| += | x+=y | x=x+y |
| -= | x-=y | x=x-y |
| *= | x*=y | x=x*y |
| /= | x/=y | x=x/y |
| %= | x%=y | x=x%y |

## Conditional Operator

This is also called a `ternary operator`. It works on three operands. The ternary operator is used to execute a set of statements when the test expression is true and another set of statements when the test expression evaluates to false.

Here, the testexpression results in a boolean value i.e. 0(true) or 1(false). **If the testexpression evaluates to:**

- **True:** expression1 before the colon `(:)` executes
- **False:** expression2 after the colon `(:)` executes

```
#include <stdio.h>
 int main(){
 int a = 5;
 int b = 10;
 int max = (a > b) ? a : b;
 printf("The maximum value is: %d\n", max);
 return 0;
}
```

**Comma Operator**

Commas are used to divide expressions in a statement and are a part of C's miscellaneous operator `(,)`.

**Sizeof Operator**

Size of operator is a Unary operator, which gives size of operand in terms of byte that occupied in the memory. An operand may be variable, constant.
It determines the length of entities, arrays and structures when their size are not known to the programmer. It is also use to allocate size of memory dynamically during execution of the program.

EXAMPLE

```
main( )
{
int sum;
float f;
printf( "%d%d" ,size of(f), size of (sum) );
}
```

# C Data Types

In C programming, data types are declarations for variables. This determines the type and size of data associated with variables. For example, int myVar;

## Basic types

Here's a table containing commonly used types in C programming for quick access.

| Type | Size (bytes) | Format Specifier |
|---|---|---|
| `int` | at least 2 | `%d` |
| `char` | 1 | `%c` |
| `float` | 4 | `%f` |
| `double` | 8 | `%lf` |
| `short int` | 2 usually | `%hd` |

| Type | Size (bytes) | Format Specifier |
|---|---|---|
| `unsigned int` | at least 2, usually 4 | `%u` |
| `long int` | at least 4, usually 8 | `%ld`, `%li` |
| `long long int` | at least 8 | `%lld`, `%lli` |
| `unsigned long int` | at least 4 | `%lu` |
| `unsigned long long int` | at least 8 | `%llu` |
| `signed char` | 1 | `%c` |
| `unsigned char` | 1 | `%c` |
| `long double` | at least 10, usually 12 or 16 | `%Lf` |

# Tokens in C

Tokens in C is the most important element to be used in creating a program in C. We can define the token as the smallest individual element in C. For `example, we cannot create a sentence without using words; similarly, we cannot create a program in C without using tokens in C. Therefore, we can say that tokens in C is the building block or the basic component for creating a program in C language.

**Classification of tokens in C**

Tokens in C language can be divided into the following categories:

- **Keywords in C**
- **Identifiers in C**
- **Strings in C**:- Strings in C are always represented as an array of characters having null character '\0' at the end of the string. This null character denotes the end of the string. Strings in C are enclosed within double quotes, while characters are enclosed within single characters. The size of a string is a number of characters that the string contains.
- **Operators in C**
- **Constant in C:-** A constant is a value assigned to the variable which will remain the same throughout the program, i.e., the constant value cannot be changed.

There are two ways of declaring constant:

- Using const keyword
- Using #define pre-processor

**Types of <u>constants in C</u>**

| Constant | Example |
|---|---|

| | |
|---|---|
| Integer constant | 10, 11, 34, etc. |
| Floating-point constant | 45.6, 67.8, 11.2, etc. |
| Octal constant | 011, 088, 022, etc. |
| Hexadecimal constant | 0x1a, 0x4b, 0x6b, etc. |
| Character constant | 'a', 'b', 'c', etc. |
| String constant | "java", "c++", ".net", etc. |

- **Special characters in C**

Some special characters are used in C, and they have a special meaning which cannot be used for another purpose.

- **Square brackets [ ]:** The opening and closing brackets represent the single and multidimensional subscripts.
- **Simple brackets ( ):** It is used in function declaration and function calling. For example, printf() is a pre-defined function.
- **Curly braces { }:** It is used in the opening and closing of the code. It is used in the opening and closing of the loops.
- **Comma (,):** It is used for separating for more than one statement and for example, separating function parameters in a function call, separating the variable when printing the value of more than one variable using a single printf statement.
- **Hash/pre-processor (#):** It is used for pre-processor directive. It basically denotes that we are using the header file.
- **Asterisk (*):** This symbol is used to represent pointers and also used as an operator for multiplication.
- **Tilde (~):** It is used as a destructor to free memory.
- **Period (.):** It is used to access a member of a structure or a union.